FIAP NUTRITION STORE: MODERNIZAÇÃO ARQUITETURAL

Nomes	RMs
Adriano Silva	RM 362980
Ana Paula da Silva Mendes	RM 361705
Gustavo Calderon	RM 364458
Leonam Pereira Dias	RM 360829
Wésley Fonseca Pivoto Dias	RM 364183
Anderson Fiuza de Paula	RM 360714

1.FIAP Nutrition Store 🏋️

A **FIAP Nutrition Store** é uma empresa do setor de nutrição esportiva e suplementação para atletas, praticantes de musculação, corredores e consumidores focados em saúde e performance. A empresa nasceu com operação física, expandiu para e-commerce e aplicativo mobile, e cresceu rapidamente impulsionada por campanhas digitais, influência de creators fitness, recorrência de compra e aumento do portfólio de produtos, como whey protein, creatina, pré-treinos, vitaminas, acessórios e kits promocionais.

Apesar do crescimento comercial, a empresa enfrenta um gargalo tecnológico semelhante ao do case original: seu sistema central foi construído como um **monólito**, sustentando catálogo, cadastro, estoque, carrinho, pedido, pagamento, promoções e integrações operacionais. Esse modelo funcionou no passado, mas agora reduz a velocidade de entrega, aumenta o risco de mudanças, dificulta o trabalho paralelo dos times e prejudica a resposta rápida ao mercado.

No contexto de suplementos, os impactos do legado ficam ainda mais visíveis:

- Campanhas promocionais precisam entrar no ar rápido;
- Picos de acesso ocorrem em lançamentos e ações com influenciadores;
- O catálogo muda com frequência;
- Há necessidade de recomendação e recorrência;
- Estoque, lote e validade exigem mais controle;
- Falhas em pagamento ou pedido impactam diretamente conversão e reputação.

A FIAP Nutrition Store decide iniciar uma **modernização arquitetural progressiva**, adotando uma arquitetura orientada a microsserviços, começando pelos domínios mais críticos para o negócio digital, mantendo coexistência temporária com o monólito durante a transição.

2. Storytelling sobre o problema e definição do tema 📖

Tema

Modernização arquitetural de um e-commerce de suplementos esportivos com foco em escalabilidade, agilidade e experiência digital.

Storytelling

A FIAP Nutrition Store cresceu rápido. O que antes era uma operação simples de varejo se transformou em uma marca digital com grande volume de acessos, campanhas semanais, promoções relâmpago, forte recorrência de compra e expectativa de experiência fluida em qualquer canal. Só que o crescimento do negócio não foi acompanhado pela evolução da arquitetura.

O sistema central, monolítico e altamente acoplado, passou a atrasar lançamentos, gerar conflitos entre times e tornar qualquer alteração arriscada. Uma mudança em pedido ou promoção exige testes amplos, coordenação entre múltiplas equipes e janelas de publicação demoradas. Enquanto isso, concorrentes digitais operam com maior velocidade, personalização e resiliência.

O problema que queremos resolver é claro: **como transformar a arquitetura da FIAP Nutrition Store para suportar crescimento, acelerar entregas e reduzir risco operacional, sem parar a operação do negócio?**

3. O que esperamos aprender com esse projeto? 💡

Esperamos aprender como:

- Evoluir de um monólito para uma arquitetura de transição mais modular;
- Identificar os domínios mais importantes para começar a decomposição;
- Equilibrar velocidade de entrega com risco arquitetural;
- Desenhar uma arquitetura que suporte crescimento de catálogo, pedidos e campanhas;
- Comunicar a solução por meio de diagramas em níveis diferentes, especialmente com C4.

4. Que perguntas precisamos que sejam respondidas? ❓

As perguntas centrais do projeto são:

1. Quais partes do monólito mais prejudicam o negócio hoje?
2. Quais domínios devem ser extraídos primeiro?
3. Como manter operação contínua durante a transição?
4. Como garantir consistência entre pedido, pagamento e estoque?
5. Como sustentar picos de acesso sem degradar a experiência?
6. Como reduzir acoplamento entre times?
7. Como proteger dados do cliente e operações financeiras?
8. Como comunicar a arquitetura de forma compreensível para negócio e tecnologia?

5. Quais são os nossos principais riscos? ⚠️

Os principais riscos são:

- Falha na transição e interrupção da operação;
- Aumento de complexidade distribuída sem ganho real;
- Inconsistência entre serviços;
- Duplicação temporária de dados e problemas de sincronização;
- Degradação de performance entre chamadas de serviço;
- Observabilidade insuficiente em ambiente distribuído;
- Resistência organizacional dos times;
- Custo de modernização acima do esperado.

6. Plano para aprender o que precisamos para responder perguntas específicas 📚

Frente 1: Negócio

Entrevistar marketing, operação, atendimento e produto para entender onde o sistema mais atrasa campanhas, promoções, cadastro de produtos e gestão de pedidos.

Frente 2: Tecnologia

Mapear o monólito atual, módulos, integrações, tabelas principais, dependências internas e pontos de maior mudança.

Frente 3: Dados

Identificar entidades críticas: cliente, produto, estoque, carrinho, pedido, pagamento, entrega e promoção.

Frente 4: Operação

Levantar incidentes recorrentes, gargalos de deploy, tempo médio para mudança e principais falhas em produção.

Frente 5: Arquitetura

Definir limites de domínio, serviços candidatos, integrações síncronas e assíncronas, banco por serviço quando aplicável e estratégia de transição.

7. Plano para reduzir riscos 🛡️

Para reduzir os riscos, o plano inclui:

- Começar por domínios de maior valor e menor dependência;
- Usar strangler pattern para extração gradual;
- Manter coexistência entre monólito e novos serviços durante a transição;
- Adotar mensageria para desacoplamento;
- Aplicar observabilidade desde o início;
- Usar feature flags e rollout progressivo;

- Definir contratos claros de API e eventos;
- Criar testes de integração e testes de contrato;
- Formalizar ADRs para decisões relevantes.

8. Quem são as partes interessadas? 🤝[1] [2]

Na FIAP Nutrition Store, os stakeholders são:

- Diretoria e patrocinadores do projeto;
- Área de negócio/e-commerce;
- Marketing digital;
- Operação logística;
- Atendimento ao cliente;
- Financeiro;
- Time de tecnologia;
- Arquitetura;
- Segurança/compliance;
- Parceiros de pagamento e entrega.

9. O que eles esperam ganhar? 🏆

As expectativas dos stakeholders incluem:

- Diretoria: crescimento com previsibilidade;
- Negócio: time-to-market menor;
- Marketing: promoções e campanhas rápidas;

- Operação: menor ruptura no fluxo de pedidos;
- Atendimento: menos falhas visíveis ao cliente;
- Tecnologia: autonomia por domínio e deploy independente;
- Arquitetura: redução de acoplamento e melhor escalabilidade;
- Financeiro: menos perda por indisponibilidade e retrabalho;
- Segurança/compliance: mais controle sobre acessos e dados.

10. Quem são os usuários? 👤

Os usuários do sistema da FIAP Nutrition Store são:

- Clientes finais do e-commerce e app;
- Atendentes de suporte;
- Analistas de catálogo;
- Operadores logísticos;
- Analistas de marketing;
- Administradores internos.

11. O que eles estão tentando realizar? 🎯

Cada tipo de usuário busca realizar tarefas específicas:

- Cliente final: encontrar suplemento, comparar, comprar, pagar e receber com confiança;
- Atendimento: localizar pedido, status e problemas rapidamente;
- Catálogo: cadastrar e atualizar produtos e kits;
- Marketing: criar campanhas, combos, descontos e vitrines;
- Operação: separar, faturar e despachar pedidos;
- Gestão: acompanhar indicadores de conversão e performance.

12. Qual o pior que pode acontecer? 💀

O pior cenário é uma migração mal conduzida gerar indisponibilidade, pedidos perdidos, pagamentos inconsistentes, ruptura de estoque, reclamações em massa e dano reputacional. Em uma empresa com forte canal digital, isso compromete receita, confiança e continuidade operacional.

13. Arquitetura inicial — modelo freeform (versão inicial) 🏗️

A visão inicial proposta é esta:

Canais

- Web SPA
- App Mobile
- Painel Admin

Entrada

- API Gateway / BFF

Núcleo de domínio

- Serviço de Catálogo
- Serviço de Carrinho
- Serviço de Pedido
- Serviço de Pagamento
- Serviço de Estoque
- Serviço de Promoção
- Serviço de Cliente
- Serviço de Notificação

Integração

- Message Broker / Event Bus

Persistência

- Banco do monólito legado
- Bancos dedicados por serviço extraído

Sistemas externos

- Gateway de pagamento
- Transportadora / antifraude / e-mail-SMS-push

Transição

- Monólito permanece atendendo partes ainda não extraídas
- Novos serviços assumem domínios prioritários gradualmente

Estratégia de Transição (Strangler Pattern & MVA)

Definimos o caminho para sair do monólito sem o risco de um "Big Bang".[3]

A. Definição da MVA (Minimum Viable Architecture)

A **MVA** para a FIAP Nutrition Store é a base mínima necessária para sustentar o primeiro microsserviço (Catálogo) coexistindo com o monólito.

Componentes da MVA:

1. **Traffic Router (API Gateway):** Um ponto único de entrada (ex: Nginx ou Kong) que decide se a requisição vai para o Monólito ou para o novo serviço.
2. **Microsserviço de Catálogo:** Primeiro domínio extraído por ter alto volume de leitura e baixo acoplamento transacional.
3. **Banco de Dados Isolado:** Instância separada para o Catálogo, garantindo o padrão *Database per Service*.
4. **Pipeline CI/CD Básico:** Automação mínima para garantir que o novo serviço seja testado e implantado sem intervenção manual.

B. Implementação do Strangler Pattern com ACL

Para garantir que o novo Serviço de Catálogo não seja "corrompido" pela semântica antiga do Monólito, implementamos uma **ACL (Anti-Corruption Layer)**.

Funcionamento da ACL entre Monólito e Catálogo:

- **A Camada de Tradução:** Quando o Monólito precisa de dados de um produto que agora "mora" no microsserviço, ele não acessa o banco novo diretamente. Ele chama uma interface na ACL.
- **Mapeamento de Domínio:** A ACL recebe o modelo de dados legado do Monólito, traduz para o novo contrato do Microsserviço (e vice-versa), garantindo que a evolução do novo serviço seja independente.
- **Sincronização via Eventos:** Durante a transição, a ACL pode usar um **Message Broker** para replicar mudanças de estoque do Monólito para o novo Catálogo em tempo quase real.

14. Descrição dos componentes 🧩

A seguir, a descrição detalhada de cada componente da arquitetura proposta:

Web SPA / App Mobile

Interfaces de consumo do cliente. Exibem catálogo, carrinho, checkout e histórico.

Painel Admin

Usado por times internos para catálogo, pedidos, promoções e suporte.

API Gateway / BFF

Camada de entrada única para frontends. Consolida chamadas, aplica autenticação, roteamento e políticas.

Serviço de Catálogo

Responsável por produtos, categorias, composição de kits, descrições e busca.

Serviço de Carrinho

Mantém o estado do carrinho e simulações de compra.

Serviço de Pedido

Orquestra a criação do pedido e seu ciclo de vida.

Serviço de Pagamento

Integra com provedor externo para autorização, captura, estorno e atualização de status.

Serviço de Estoque

Controla saldo disponível, reserva, baixa e eventualmente lote/validade.

Serviço de Promoção

Aplica regras de cupom, desconto, combo, frete promocional e campanhas.

Serviço de Cliente

Gerencia cadastro, autenticação federada, preferências e histórico.

Serviço de Notificação

Envia e-mail, SMS, push e eventos transacionais.

Message Broker

Desacopla produtores e consumidores; propaga eventos como PedidoCriado, PagamentoAprovado e EstoqueReservado.

Monólito Legado

Permanece temporariamente com os domínios ainda não extraídos, reduzindo risco de big bang.

Bancos dedicados

Suportam autonomia por domínio e isolamento de mudanças.

Sistemas externos

Pagamento, logística, antifraude e comunicação.

15. Requisitos importantes e por quê? ✅

Os requisitos cruciais para o sucesso do projeto são:

9. Escalabilidade: Campanhas e picos de acesso exigem crescimento seletivo dos componentes mais demandados.
10. Disponibilidade: A loja digital precisa operar com alta continuidade, especialmente em checkout e consulta de pedido.
11. Desempenho: Busca, vitrine, carrinho e checkout devem responder rápido para evitar abandono.
12. Segurança: Dados pessoais e transações financeiras exigem autenticação forte, rastreabilidade e proteção de dados.
13. Manutenibilidade: O principal problema do case é a dificuldade de mudar. A arquitetura precisa permitir evolução com menor acoplamento.
14. Observabilidade: Ambientes distribuídos exigem logs centralizados, métricas e tracing.
15. Resiliência: Falhas de parceiro externo não podem derrubar toda a jornada.
16. Implantação independente: Times precisam publicar mudanças sem depender de um deploy único do sistema inteiro.

Refinamento dos requisitos significativos (ASRs)[4]

Atributo	Fonte	Estímulo	Artefato	Ambiente	Resposta	Medida de Resposta
Escalabilidade	Usuário final	Pico de acessos via campanha de influenciador	Serviço de catálogo	Produção (carga alta)	O sistema escala horizontalmente instâncias do serviço	Processar 500 req/s com latência média < 200ms.
Disponibilidade	Infraestrutura	Falha crítica no banco de dados de pedidos	Serviço de pedidos	Operação normal	O sistema ativa modo de leitura ou failover e notifica o time	Downtime máximo de 30 segundos; log gerado em 3s.
Desempenho	Cliente	Consulta de vitrine personalizada	Serviço de Produto / Cache	Operação normal	Renderização dos dados na UI do cliente	Tempo de resposta (TTFB) inferior a 100ms para 95% das requisições.
Segurança	Atacante externo	Tentativa de acesso via força bruta / Injeção	API Gateway	Rede pública	O sistema bloqueia a origem e gera alerta de segurança	Bloqueio de IP após 5 tentativas inválidas

						em 1 minuto.
Manutenibilidade	Desenvolvedor	Necessidade de alterar regra de frete	Microsserviço de Promoção	Tempo de design	Alteração realizada e deploy feito via CI/CD	Impacto zero nos outros serviços; tempo de build/deploy < 10 min.
Resiliência	Sistema externo	Indisponibilidade do Gateway de Pagamento	Serviço de Pagamento	Checkout ativo	Ativação de Circuit Breaker e retentativa assíncrona	Interface informa erro amigável em < 1s sem travar o thread.
Observabilidade	Operações	Erro intermitente em fluxo de checkout	Rastreamento Distribuído	Produção	Identificação da causa raiz através de correlação de IDs	Rastreio completo (end-to-end) disponível em menos de 1 minuto no Dashboard.

16. Sobre o que o diagrama ajuda você a raciocinar/pensar? 🤔

O diagrama é uma ferramenta essencial para:

- Fronteiras de domínio;
- Separação de responsabilidades;
- Dependências entre serviços;
- Pontos de acoplamento forte;
- Fluxo principal de compra;
- Eventos de integração;
- Prioridades de transição;
- Riscos de consistência e disponibilidade.

17. Quais são os padrões essenciais no diagrama? ✨

Os padrões arquiteturais essenciais a serem representados são:

- Strangler Pattern para migração incremental;
- API Gateway / BFF para frontends;
- Database per Service onde houver extração madura;
- Event-Driven Architecture para integração desacoplada;
- SAGA/Coreografia ou orquestração leve no fluxo de pedido-pagamento-estoque;
- Observabilidade centralizada;
- Deploy independente por serviço.

18. Existem padrões ocultos? 🕵️

Sim. Alguns padrões não aparecem explicitamente de primeira, mas estão implícitos e são cruciais:

- Anti-corruption layer entre monólito e novos serviços;
- Eventual consistency;
- Retries, circuit breaker e idempotência;
- Cache em leitura para catálogo;
- Contratos de eventos e integração.

19. Qual é o Metamodelo? ⚛️

O metamodelo é a estrutura conceitual que organiza os elementos da arquitetura e seus relacionamentos. Neste caso, ele pode ser entendido como:

Atores → **Canais** → **Serviços/Containers** → **Componentes internos** → **Dados** → **Integrações/Eventos** → **Sistemas externos**

Ou, em linguagem mais prática:

- Pessoas e sistemas externos interagem com a solução;
- A solução é composta por containers;
- Containers são compostos por componentes;
- Componentes manipulam dados e trocam mensagens.

20. Pode ser discernido no diagrama único? 🖼️

Parcialmente. Um único diagrama pode sugerir o metamodelo, mas não é o ideal para explicá-lo completamente. Para comunicação eficaz, é melhor separar em níveis:

- Contexto;
- Container;
- Componente.

Isso está aderente à abordagem da aula e ao uso do C4.

21. O diagrama está completo? 💡

Não totalmente, e isso é aceitável na versão inicial. Ele está suficientemente completo para discussão arquitetural, mas ainda pode evoluir com:

- Fluxos de erro;
- Segurança;
- Observabilidade;
- Estratégia de dados;
- Integrações operacionais;
- Regras de transição.

22. Poderia ser simplificado e ainda assim ser eficaz? 🗣️

Sim. Para comunicação executiva, ele pode ser simplificado para:

- Canais;
- Gateway;
- Serviços centrais;
- Monólito legado;
- Mensageria;
- Sistemas externos.

Já para o time técnico, o detalhamento maior é útil.

23. Houve alguma discussão importante que vocês tiveram como equipe? 🚧

Uma discussão importante seria: **migrar tudo de uma vez ou modernizar gradualmente?**

A conclusão recomendada é migração gradual, porque reduz risco operacional e se alinha melhor ao contexto do case.

Outra discussão relevante: **pedido ou catálogo deve ser o primeiro domínio a sair?**

Pedido costuma ter alto valor estratégico, mas também maior criticidade transacional. Catálogo é mais simples, porém pode gerar menos impacto imediato. A decisão depende do objetivo do grupo.

24. Que decisões sua equipe teve dificuldade para tomar? ❓

As decisões que apresentaram maior dificuldade foram:

- Qual domínio extrair primeiro;
- Quanto manter no monólito temporariamente;
- Quando usar integração síncrona vs assíncrona;
- Quando separar banco por serviço;
- Qual fronteira correta entre pedido, pagamento e estoque.

25. Que decisões foram tomadas sob incerteza? 🎲

Algumas decisões foram tomadas com base em estimativas e sob incerteza, incluindo:

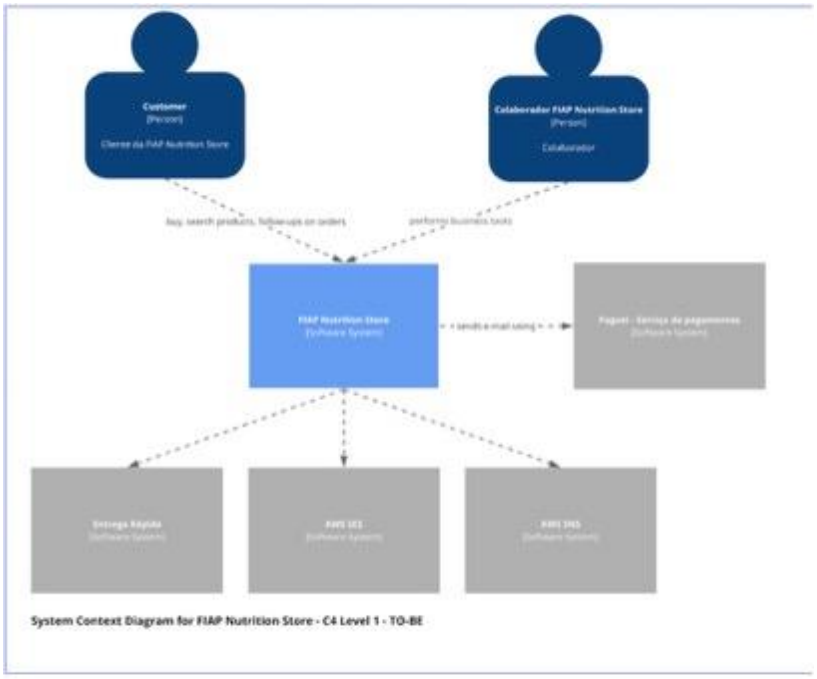
- Estimativa de volume futuro;
- Impacto real da decomposição na produtividade;
- Nível de maturidade dos times para operar microsserviços;
- Escolha da estratégia de consistência distribuída;
- Custo total da transição.

26. Houve algum ponto de decisão sem retorno? ↩️

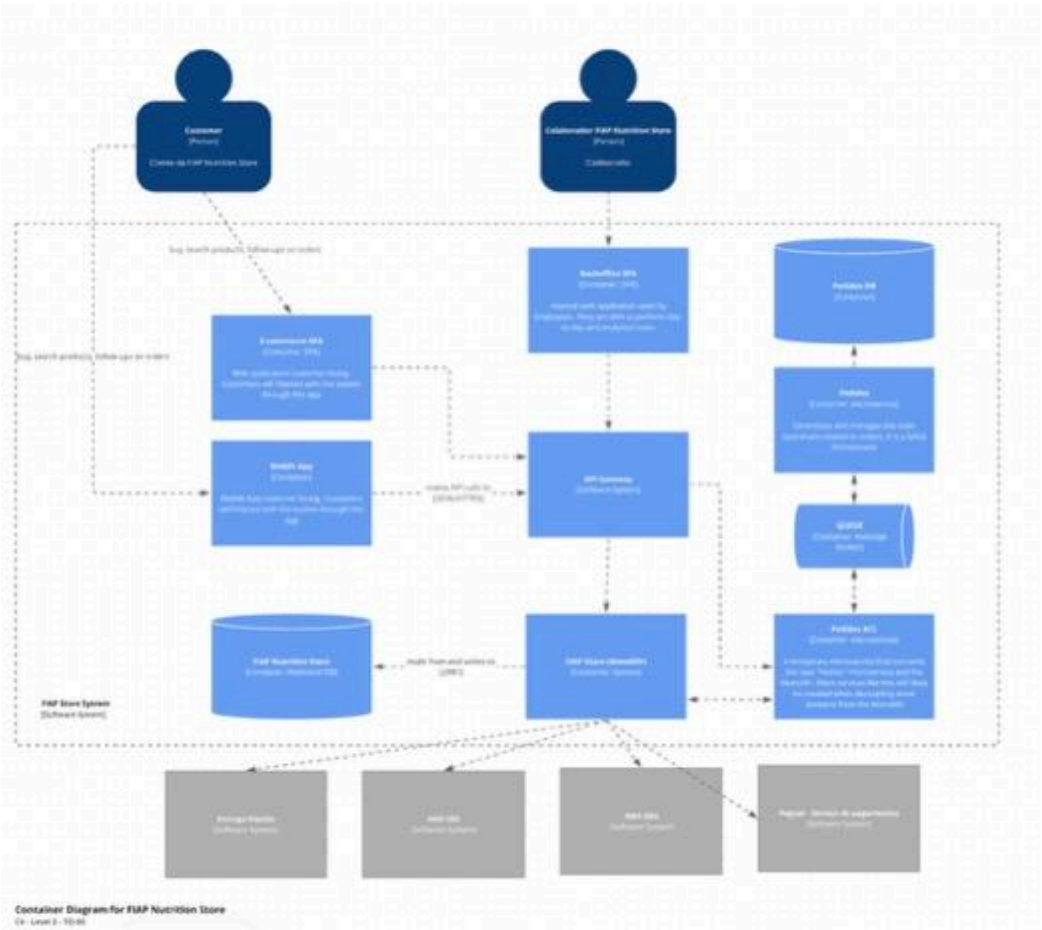
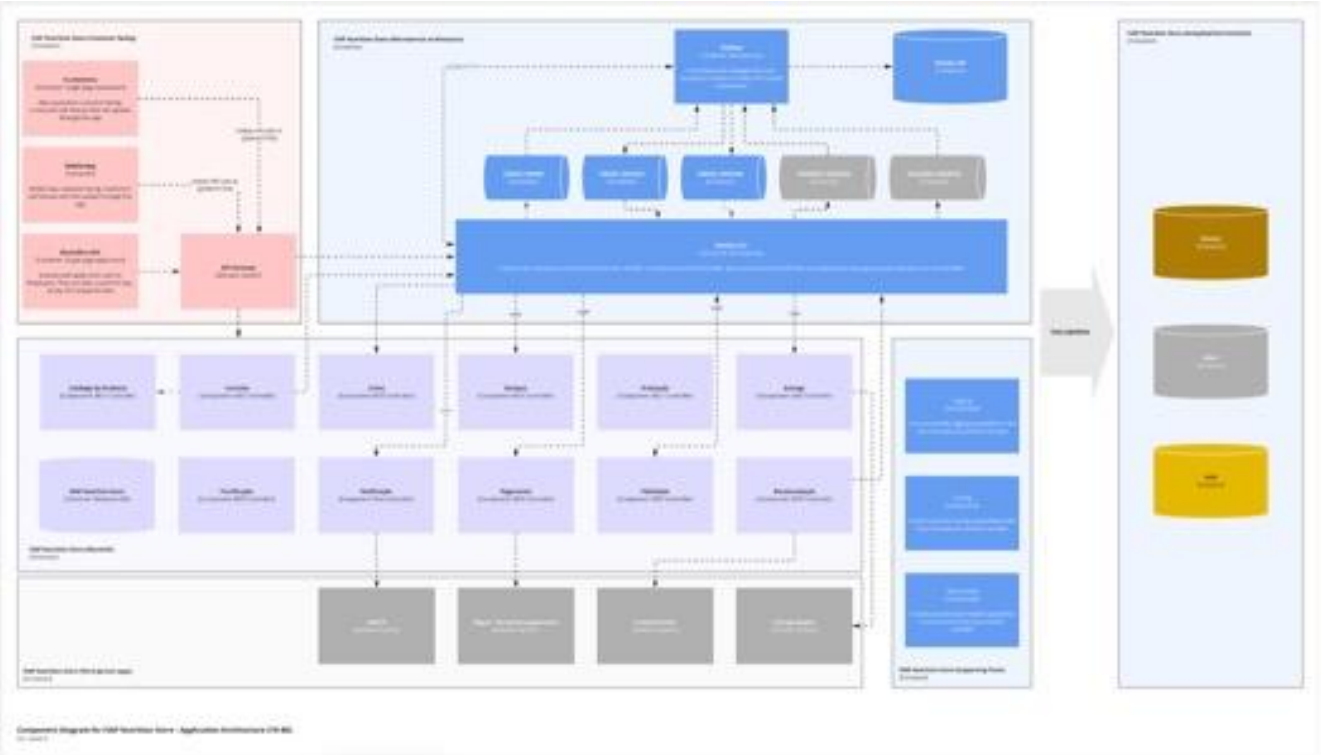
Sim: **adotar modernização incremental em vez de reescrita total**.

Depois que a empresa investe em domínio-alvo, contratos, eventos, pipeline e novos serviços, voltar para um modelo de reescrita big bang tende a se tornar inviável.

27. Nível Contexto



28. Nível Container



29. Nível Componente